

AMDARI

Cloud Security Internship Programme

Implementing Cloud Threat Detection and Automated Response with Microsoft Sentinel

For CloudScale Logistics

Implementation Report | CloudScale Logistics | Microsoft Azure, East US

Field	Detail
Analyst	Chinaza Nnamani
Client	CloudScale Logistics (mid-market freight and supply chain)
Engagement	Microsoft Sentinel SIEM and SOAR, two-week proof of concept
Primary region	East US
Frameworks	NIST Cybersecurity Framework, MITRE ATT&CK, SOC 2 Type II, ISO 27001

1. Executive Summary

This engagement establishes CloudScale Logistics' proactive cloud security operations capability. By transitioning the organization from scattered, reactive monitoring tools to a unified threat detection and response model, we are deploying Microsoft Sentinel to provide centralized visibility across the entire digital estate.

The implementation leverages Microsoft Sentinel, A cloud-native SIEM/SOAR platform to ingest, correlate, and act upon security telemetry from cloud workloads, on-premises infrastructure, and third-party services. The deployment strategy is aligned with the NIST Cybersecurity Framework (Detect, Respond, Recover) and is structured around three foundational pillars:

- **Data Ingestion:** Centralized collection of security telemetry.
- **Analytics & Detection:** Proactive identification of correlated threat activity.
- **Automation & Orchestration:** Automated, rapid response to security incidents.

Key Transformations & Outcomes:

- **Unified Visibility:** We consolidated core Azure data sources into a single Log Analytics Workspace, ensuring 100% of Azure actions are centrally searchable. Previously, administrative activity was scattered across multiple disconnected portals.
- **Rapid Incident Response:** We built an automated playbook that delivers structured incident notifications to the security team within two minutes of an alert. This drastically improves the previous baseline, which took over four hours to move a suspected credential compromise from alert to a containment decision.
- **Compliance Resolution:** This new implementation directly addresses and resolves the ISO 27001 audit finding that originally initiated the project.

2. Project Context

CloudScale Logistics moved its core platforms to Azure over the eighteen months before this engagement. The migration delivered the scalability the business wanted, but it also widened the attack surface across multiple subscriptions while the in-house team remained skilled mainly in on-premises infrastructure. Three problems followed from this.

- **No unified visibility.** Activity logs, Defender alerts and VM events lived in separate places and were never consolidated, so an attacker could move between subscriptions without raising a timely alert.
- **Manual, slow response.** With no correlation and no automation, a small team triaged hundreds of raw alerts by hand and escalated by email. One real incident took more than four hours to reach a containment decision.
- **An audit gap.** ISO 27001 auditors issued a non-conformance finding because the client could not demonstrate a documented detection and response process for its cloud systems.

Goal: The goal of the proof of concept was therefore narrow and concrete: prove that a centralized, automated detection and response capability can be stood up quickly in Azure, and produce the evidence an auditor needs to close the finding.

3. Solution Architecture

The design follows three pillars that map to the Detect, Respond and Recover phases of the NIST Cybersecurity Framework: collect the data, identify the threats, and respond automatically. Data flows in one direction through the platform, which makes the system easy to reason about and easy to audit.

Data flow (one line)

Azure data sources -> Log Analytics Workspace -> Sentinel analytics rules -> Incident -> Logic App playbook -> Team notification

In plain terms: Azure Activity logs and Microsoft Defender for Cloud feed raw security telemetry into the Log Analytics Workspace named **LAW-Sentinel-CloudScale**. Microsoft Sentinel sits on top of that workspace and runs analytics rules written in Kusto Query Language (KQL) against the incoming data. When a rule matches, Sentinel raises an incident. The incident triggers a Logic App playbook named **Playbook-incidentNotification-Email**, which formats the details and posts them to the security team members email. Every stage leaves a record in the workspace, which is what gives the auditor a full trail.

3.1 Component roles

Component	Role in the design
Log Analytics Workspace	Central store and query engine for all security telemetry. Named LAW-Sentinel-CloudScale , deployed in resource group RG-Sentinel-PoC .
Microsoft Sentinel	The SIEM and SOAR layer. Runs analytics rules, manages incidents, hosts playbooks.
Azure Activity logs	Control-plane data source. Records who did what, when, and from where across the subscription.
Microsoft Defender for Cloud	Workload data source. Supplies enriched alerts for VMs, storage and other resources, mapped to MITRE ATT&CK.
Azure Logic Apps	The automation engine behind playbooks. Low-code, fires on incident creation.
Kusto Query Language (KQL)	The query language used to write detection rules and verify data.

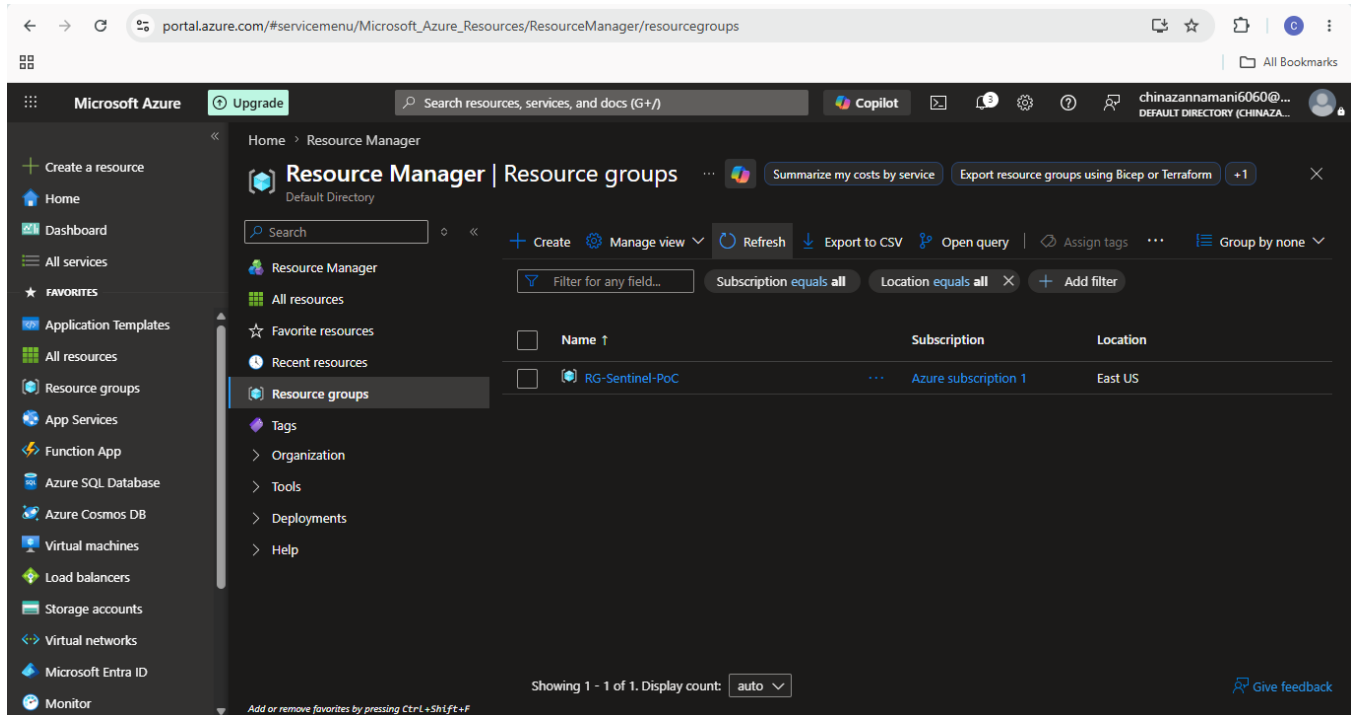
4. Implementation

The build followed the three pillars in order, because each one depends on the last. You cannot detect without data, and you cannot automate a response to an incident that detection never raises. Resource names below are the real names used, so that this section can be used to

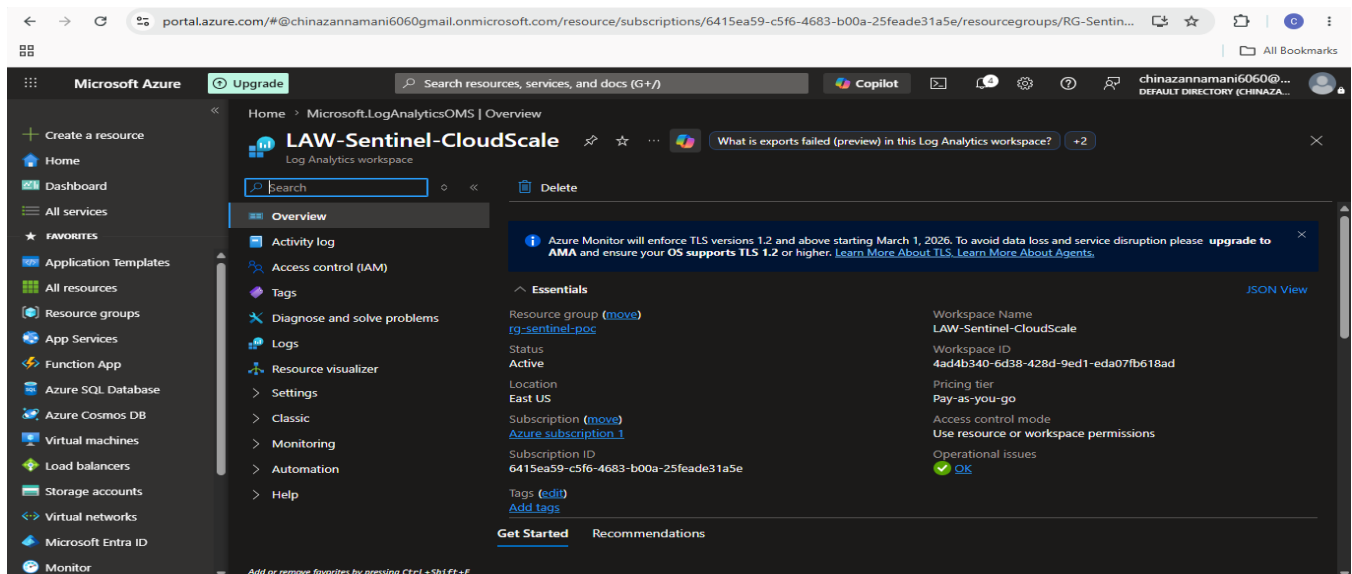
rebuild the environment exactly.

4.1 Foundation

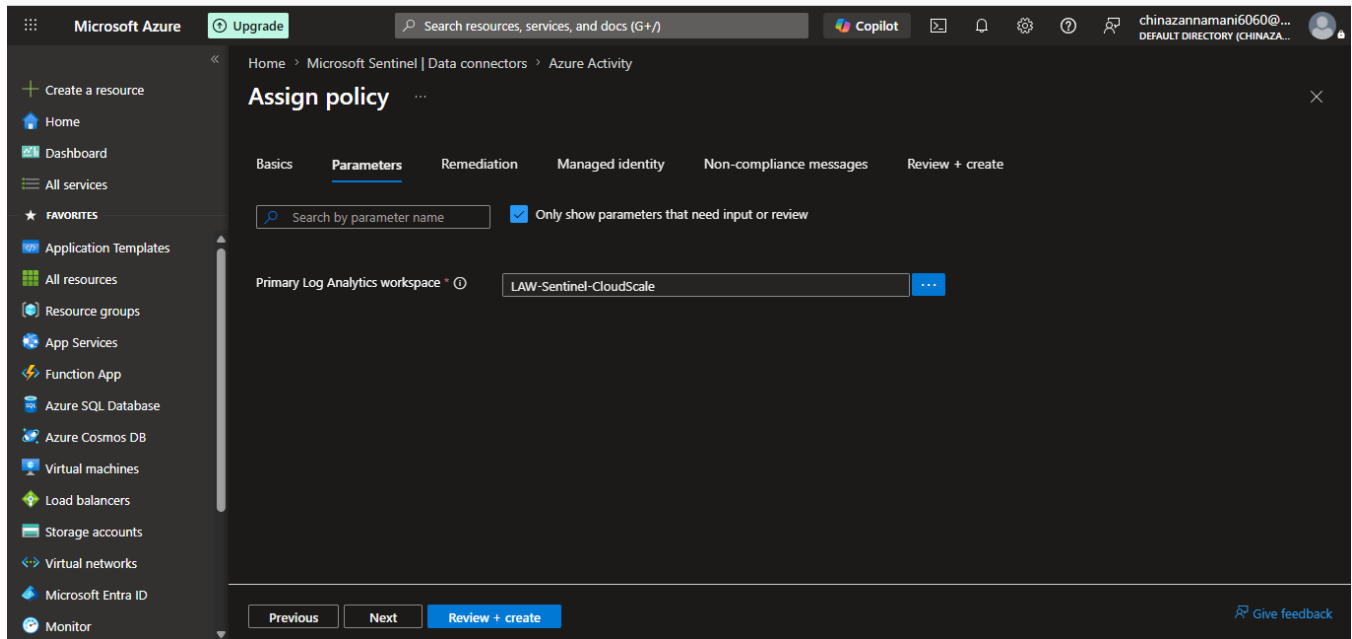
We created a single resource group, RG-Sentinel-PoC, in East US to hold every resource. Keeping all project resources in one group makes cost tracking and clean-up simple, which matters for a time-boxed proof of concept.



Inside it we deployed the Log Analytics Workspace LAW-Sentinel-CloudScale on the pay-as-you-go tier with retention set to ninety days. Ninety days is the floor required to satisfy the client's compliance obligation; we recommend one hundred and eighty days for production if budget allows.

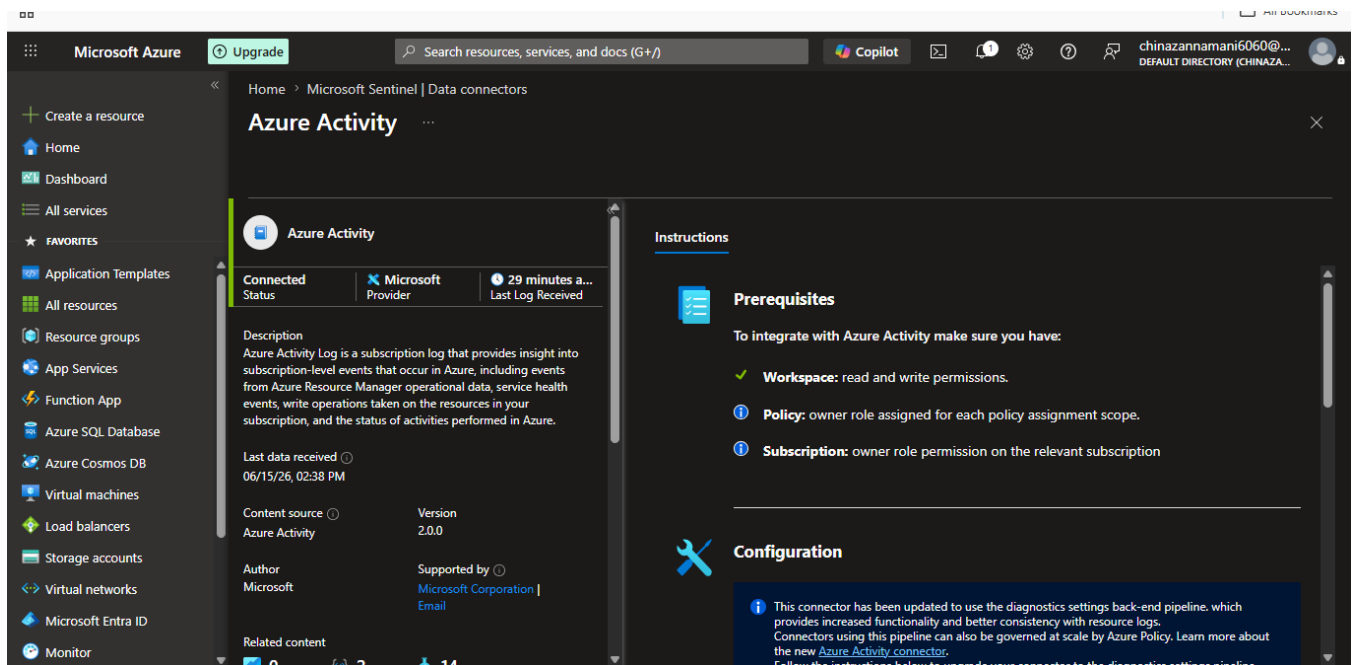


We then enabled Microsoft Sentinel on that workspace, which activates the SIEM and SOAR features on top of the raw data store.

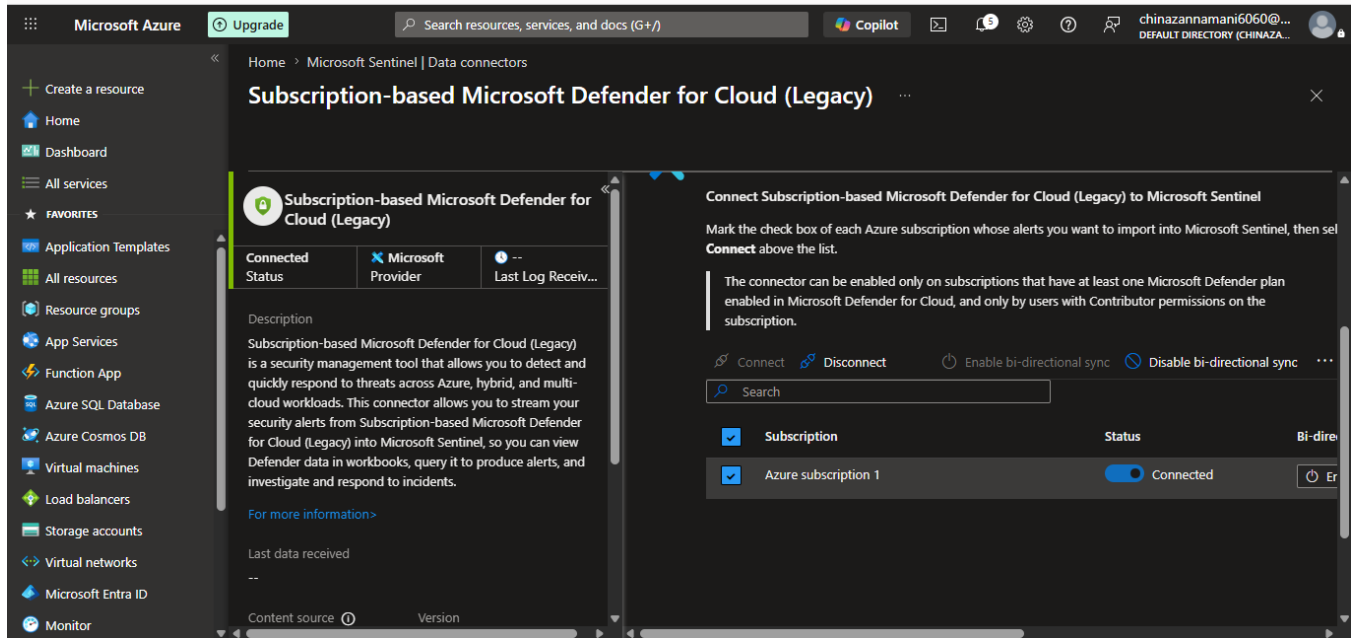


4.2 Data ingestion

Two data connectors were enabled. The Azure Activity connector brings in every control-plane operation across the subscription, which is the data our custom detection depends on.

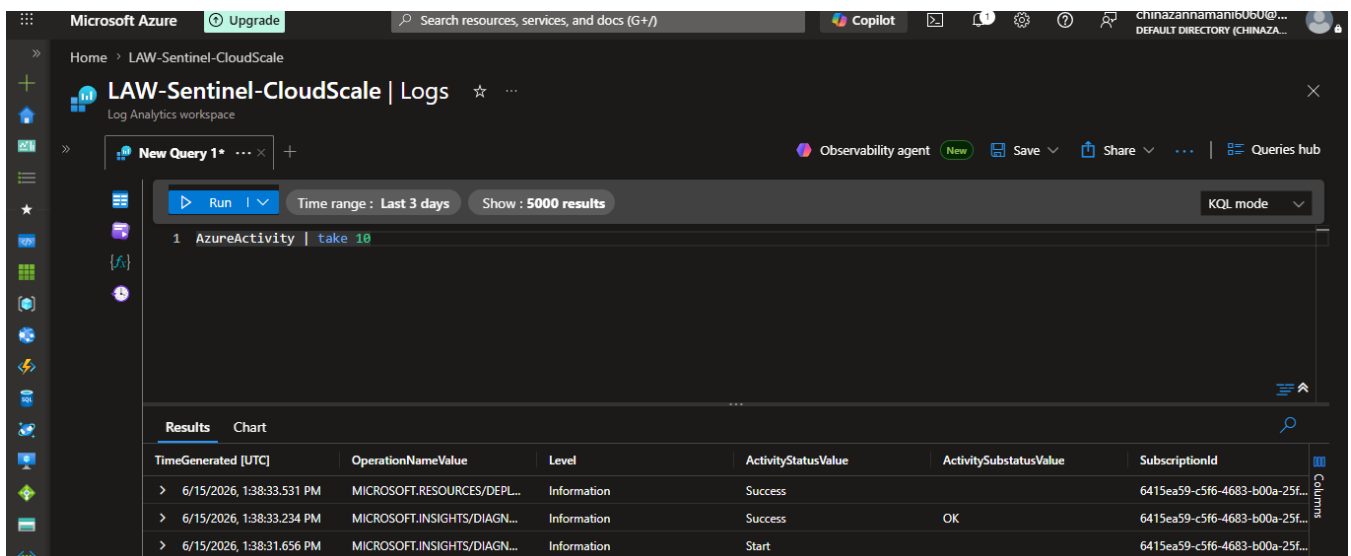


The Microsoft Defender for Cloud connector brings in pre-analysed workload alerts; we enabled the free foundational tier and deliberately deferred the paid Defender plans for the proof of concept to keep cost near zero, noting that production should enable the paid plans for deeper coverage.



Before building any detection, we confirmed that data was actually flowing. This verification step is not optional: a rule built on an empty table will silently never fire. The two queries below returned populated results within thirty minutes of enabling the connectors.

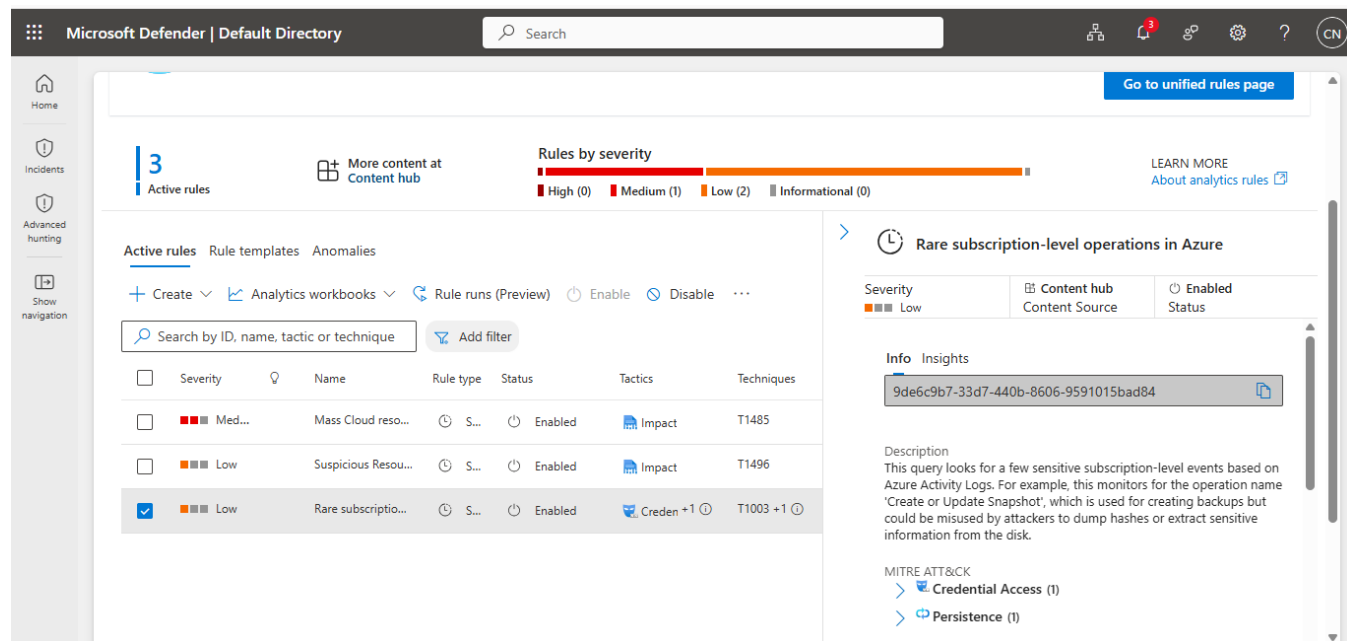
```
AzureActivity  
| take 10
```



The query returned recent, populated rows, which confirmed the ingestion pillar was complete and it was safe to build detections on top.

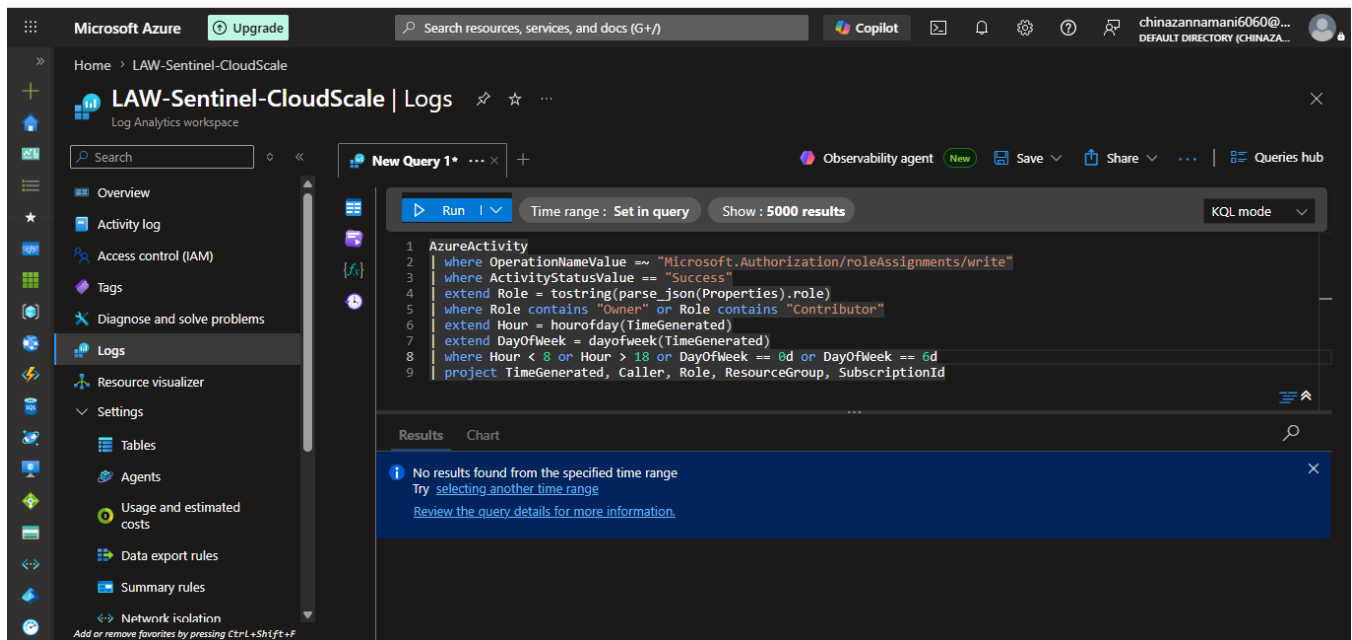
4.3 Detection engineering

We enabled Four analytics rules in total: a set of built-in scheduled rules for common attack patterns, plus one custom rule written specifically for this client. The built-in rules cover Mass Cloud resource deletions, Rare subscription-level operations, and Suspicious resource deployment. Each was mapped to its MITRE ATT&CK tactic so that incidents arrive with attacker context attached, not just a raw alert.



The custom rule is the most instructive part of the build, so it is worth walking through. The client's real risk is an insider or a compromised account quietly granting itself high privilege. We therefore wrote a rule that fires when a high-privilege role (Owner or Contributor) is assigned outside business hours, which is a common signature of unauthorised privilege escalation. The first version read like this:

```
AzureActivity
| where OperationNameValue == "Microsoft.Authorization/roleAssignments/write"
| where ActivityStatusValue == "Success"
| extend Role = tostring(parse_json(Properties).role)
| where Role contains "Owner" or Role contains "Contributor"
| extend Hour = hourofday(TimeGenerated)
| extend DayOfWeek = dayofweek(TimeGenerated)
| where Hour < 8 or Hour > 18 or DayOfWeek == 0d or DayOfWeek == 6d
| project TimeGenerated, Caller, Role, ResourceGroup, SubscriptionId
```



This is a clear statement of intent, and it is the right place to start. It reads as: find successful role-assignment writes, keep the high-privilege ones, work out the hour and day, and keep only those that fall outside the Monday to Friday, 08:00 to 18:00 window. The final line projects just the columns an analyst needs to triage.

Lesson learned, carried into the rule

During testing the first version did not fire on a real test assignment. Inspecting a sample event showed why: in the AzureActivity table the role is recorded as a role under the "Authorization" json tag, so the text match never hit. The fix was to match on the Authorization json values instead of Properties. The hardened version is below

```
AzureActivity
| where OperationNameValue == "MICROSOFT.AUTHORIZATION/ROLEASSIGNMENTS/WRITE"
| where ActivityStatusValue == "Success"
| extend Role = tostring(parse_json(Authorization).evidence)
| where Role contains "Owner" or Role contains "Contributor"
| extend Hour = hourofday(TimeGenerated)
| extend DayOfWeek = dayofweek(TimeGenerated)
| where Hour < 8 or Hour > 18 or DayOfWeek == 0d or DayOfWeek == 6d
| project TimeGenerated, Caller, Role, ResourceGroup, SubscriptionId
```

The screenshot shows the Microsoft Sentinel Logs interface. A KQL query is entered in the 'New Query 1' tab. The query filters for 'AzureActivity' where 'OperationNameValue' is 'MICROSOFT.AUTHORIZATION/ROLEASSIGNMENTS/WRITE', 'ActivityStatusValue' is 'Success', and the role contains 'Owner' or 'Contributor'. It extends the results with 'Role', 'Hour', and 'DayOfWeek', and projects 'TimeGenerated', 'Caller', 'Role', 'ResourceGroup', and 'SubscriptionId'.

The results table shows the following data:

TimeGenerated [UTC]	Caller	Role	ResourceGroup	SubscriptionId
6/19/2026, 2:56:29.656 AM	chinazannamani6060@gmail.c...	Owner	RG-SENTINEL-POC	6415ea59-
6/19/2026, 2:33:34.534 AM	chinazannamani6060@gmail.c...	Owner	RG-SENTINEL-POC	6415ea59-
6/19/2026, 2:09:14.361 AM	chinazannamani6060@gmail.c...	Owner	RG-SENTINEL-POC	6415ea59-
6/19/2026, 1:08:28.373 AM	chinazannamani6060@gmail.c...	Owner	RG-SENTINEL-POC	6415ea59-
6/18/2026, 10:41:14.311 PM	chinazannamani6060@gmail.c...	Owner	RG-SENTINEL-POC	6415ea59-
6/18/2026, 9:11:24.468 PM	chinazannamani6060@gmail.c...	Owner	RG-SENTINEL-POC	6415ea59-

The rule was configured to run every five minutes, look back over the last hour, alert when the result count is greater than zero, map the Caller field to the Account entity, and create an incident from each alert. It was mapped to the MITRE ATT&CK tactic Privilege Escalation.

The screenshot shows the 'Analytics rule wizard - Create a new Scheduled rule' interface in Microsoft Defender. The wizard is in the 'Review + create' step. The configuration is as follows:

Field	Value
Rule frequency	Run query every 5 minutes
Rule period	Last 1 hour data
Rule start time	Automatic
Rule threshold	Trigger alert if query returns more than 0 results
Event grouping	Group all events into a single alert
Suppression	Not configured
Entity mapping	Entity 1: Account Identifier: Name, Value: Caller

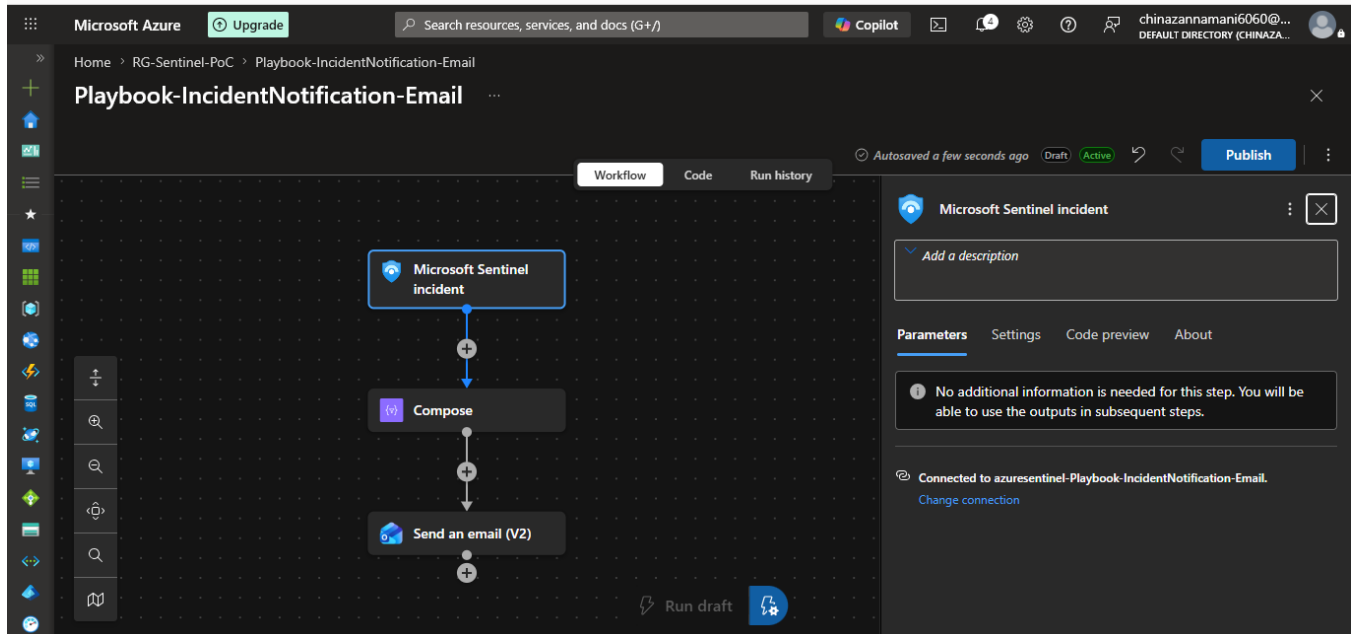
The wizard includes a progress bar on the left with steps: General, Set rule logic, Incident settings, Automated response, and Review + create. The 'Review + create' step is currently active. Buttons for '< Previous', 'Save', and 'Cancel' are at the bottom.

4.4 Incident grouping

Without grouping, a single attack produces many disconnected alerts and buries the signal. We enabled grouping so that related alerts sharing the same account or IP address within a twenty-four hour window collapse into one incident. This is the difference between an analyst triaging fifty data points and an analyst investigating one coherent story.

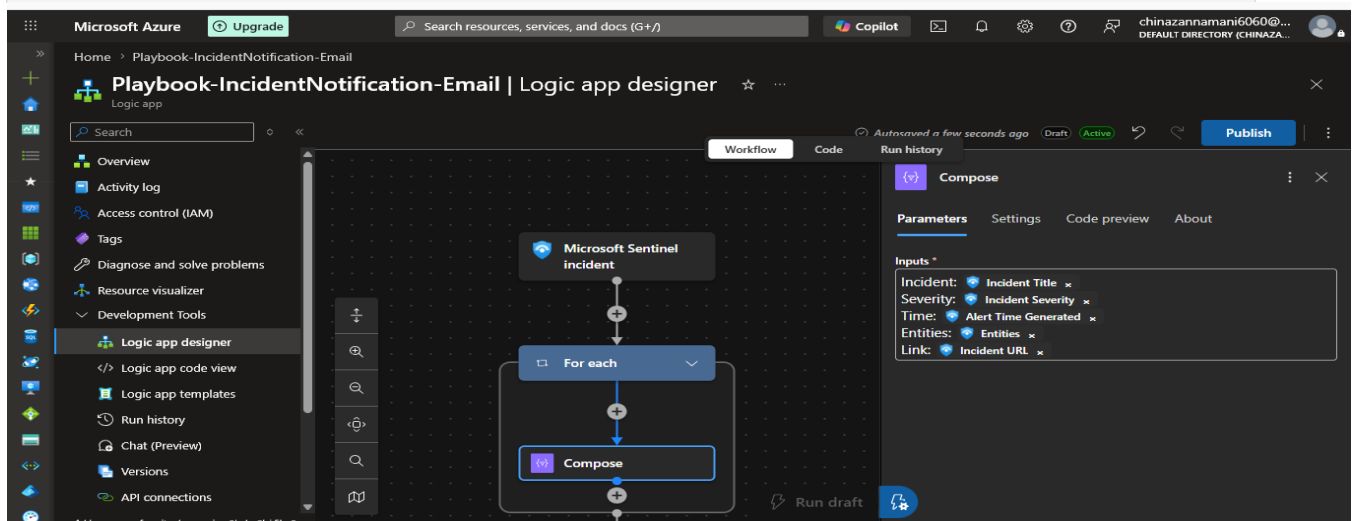
4.5 Automation

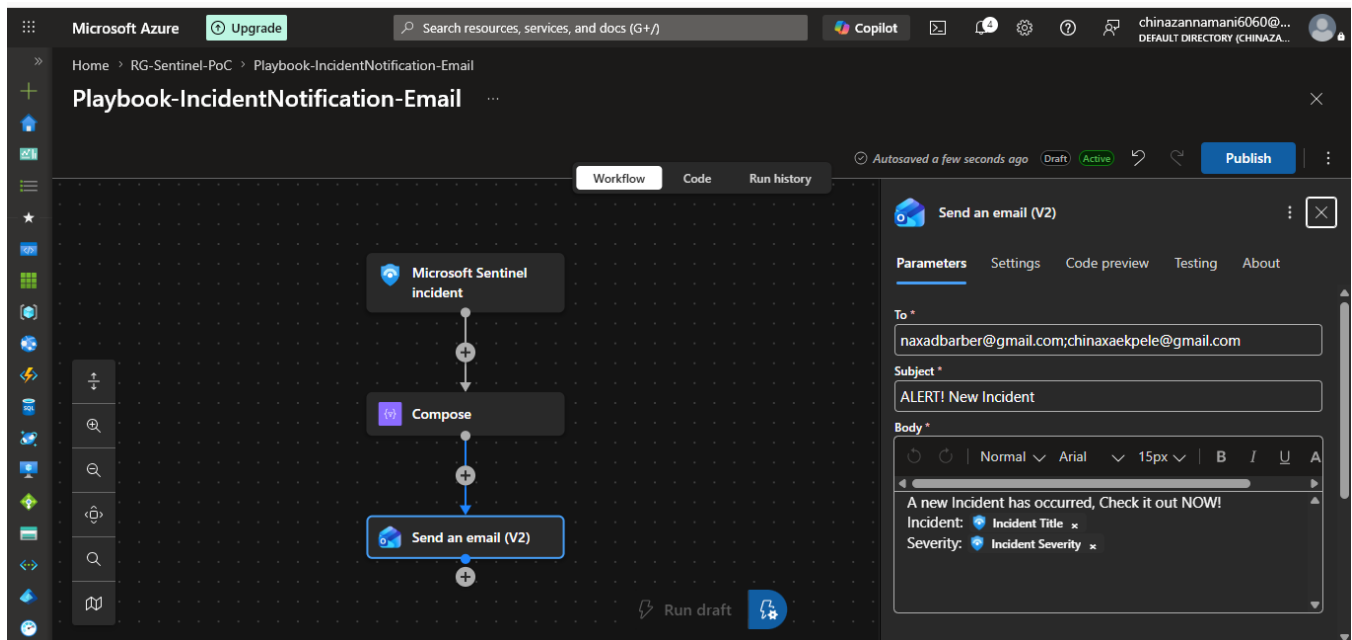
The final pillar closes the loop between detection and response. We built an Azure Logic App playbook triggered by Sentinel's incident-creation event.



The playbook composes a structured summary and posts it to the security team members' email, so the team learns of a high-priority incident in minutes rather than on the next manual review cycle. The message carries everything an analyst needs to begin work without opening the portal first.

```
Incident: [IncidentTitle]
Severity: [Severity]
Time: [TimeGenerated]
Entities: [Entities]
Link: [IncidentUrl] // deep link straight to the Sentinel incident
```





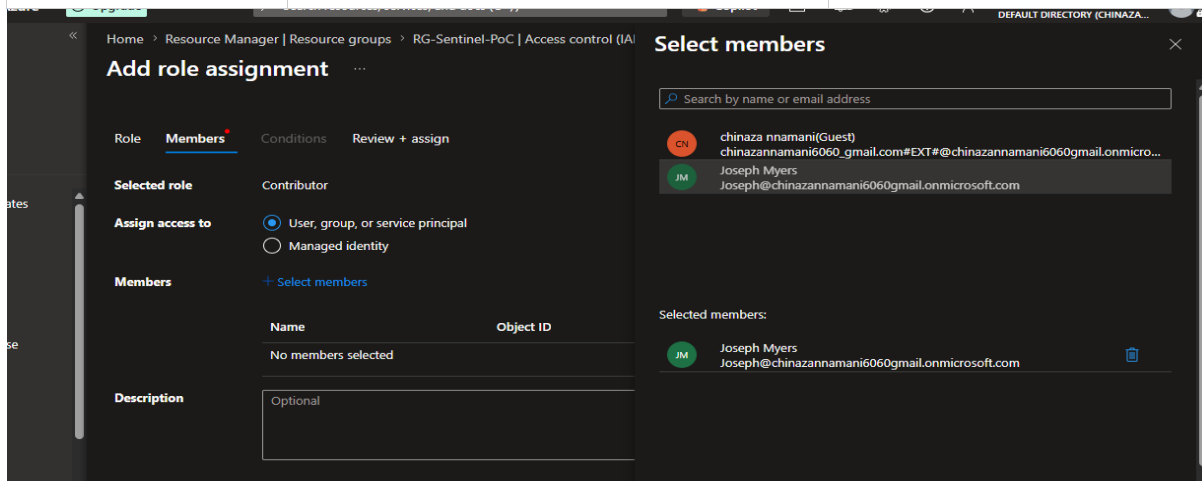
The playbook was then attached to the custom analytics rule on its automated-response tab, so that detection and notification became a single automatic chain with no human step between them. The same playbook can be attached to any other rule later without rebuilding it.

5. Testing and Validation

A capability that has not been tested end to end does not exist. We ran a single controlled test that exercises every stage of the design at once, and recorded what happened with timestamps. The test simulates the exact threat the custom rule is built for.

Method: a high-privilege role was assigned to a test account outside business hours (the rule's look-back window was temporarily adjusted so the event fell inside the trigger conditions during a working session). We then watched the chain fire.

Stage	What we observed	Evidence
Role assigned	Contributor role granted to the test account	AzureActivity row for the roleAssignments/write operation



Stage	What we observed	Evidence
Rule fired	Custom rule matched within one run cycle	Alert visible in Logs and in the rule run history

The screenshot shows the Microsoft Azure portal interface. On the left, the 'RG-Sentinel-PoC | Activity log' is selected. The main pane displays the 'List Trigger Callback URL' activity log entry for the resource group 'RG-Sentinel-PoC'. The entry details include the resource path, operation name, time stamp, and event initiated by 'Azure Security Insights'.

RG-Sentinel-PoC | Activity log

Search resources, services, and docs (G+)

Home > Resource Manager | Resource groups > RG-Sentinel-PoC

Activity log

Looking for Log Analysis

Management Group : None

Resource group : RG-Sentinel-PoC

37 items.

Operation name

- List Trigger Callback URL
- Create role assignment
- Delete role assignment
- Trigger Run
- Set Workflow
- Write Drafts
- Write Drafts

List Trigger Callback URL

Fri Jun 19 2026 03:56:39 GMT+0100 (West Africa Time)

+ New alert rule

Summary JSON Change history (Change history is not applicable to this type of activity log)

Resource: /subscriptions/6415ea59-c5f6-4683-b00a-25feade31a5e/resourceGroups/RG-Sentinel-PoC/providers/Microsoft.Logic/workflows/Playbook-IncidentNotification-Email/triggers/Microsoft_Sentinel_Incident

Operation name: List Trigger Callback URL

Time stamp: Fri Jun 19 2026 03:56:39 GMT+0100 (West Africa Time)

Event initiated by: Azure Security Insights

Stage	What we observed	Evidence
Incident created	A single incident raised, correctly grouped	Incidents blade shows the new incident

The screenshot shows the Microsoft Sentinel 'Incidents' blade. The top section displays summary statistics: 7 Open incidents, 7 New incidents, and 0 Active incidents. Below this, a table lists incidents with columns for Severity, Incident number, Title, and Alerts. The first incident is 'Privileged Role Assignment Outside Business Ho...' with incident number 6 and 1 alert. The right pane shows details for the selected incident, including its status (Unassigned), severity (Medium), and evidence (2 Events, 1 Alert, 0 Bookmarks).

Microsoft Sentinel | Incidents

Selected workspace: 'law-sentinel-cloudscale'

Search by ID, title, tags, owner or product

Severity: All

Auto-refresh incidents

Open incidents by severity

- High (0)
- Medium (7)
- Low (0)
- Informational (0)

Severity	Incident number	Title	Alerts
Medium	7	Privileged Role Assi...	1
Medium	6	Privileged Role Assi...	1
Medium	5	Privileged Role Assi...	1
Medium	4	Privileged Role Assi...	1
Medium	3	Privileged Role Assi...	1

Privileged Role Assignment Outside Business Ho...

Incident number 6

Investigate in Microsoft Defender XDR

Unassigned Owner

New Status

Medium Severity

Alert product names

- Microsoft Sentinel

Evidence

- 2 Events
- 1 Alerts
- 0 Bookmarks

Last update time: 06/19/26, 03:56 AM

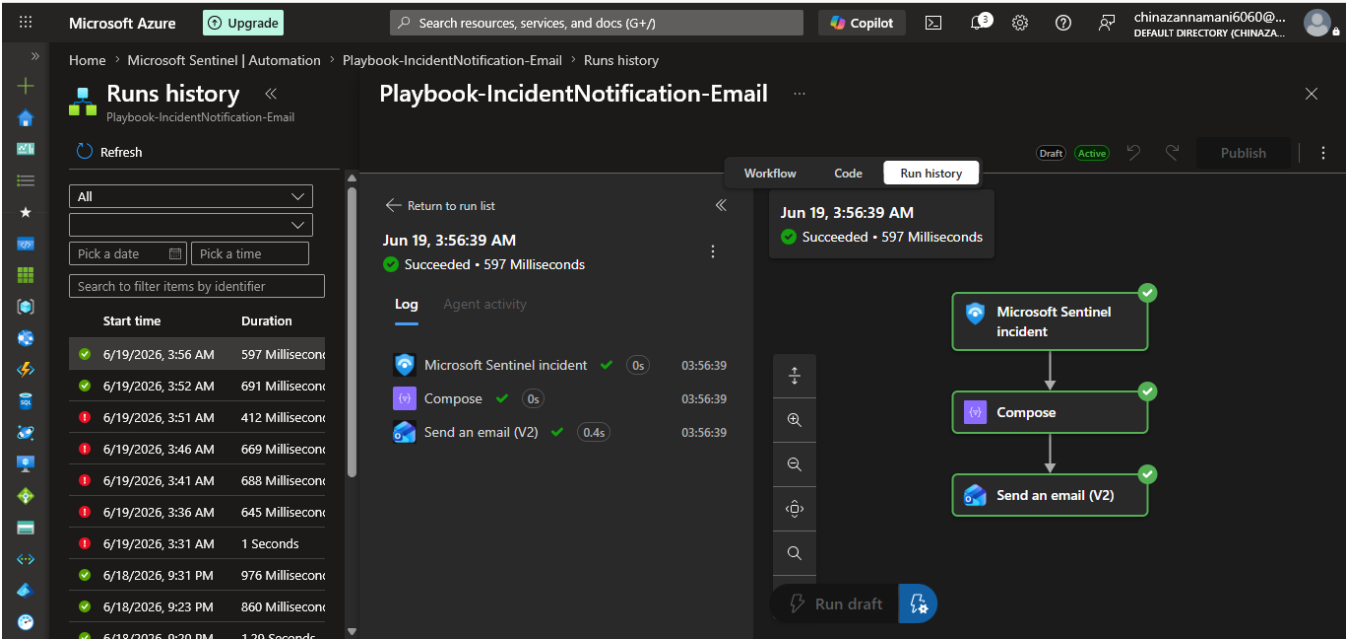
Creation time: 06/19/26, 03:56 AM

Entities (1)

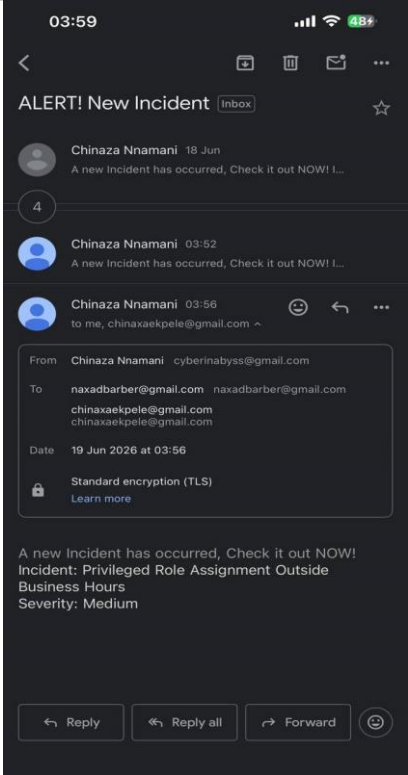
- chinazannama...

View full details >

Stage	What we observed	Evidence
Playbook ran	Logic App executed without error	Logic App run history shows a successful run



Stage	What we observed	Evidence
Team notified	Formatted message delivered with a working deep link	Notification received in Email



The full chain, from role assignment to delivered notification, completed in under five minutes. It completed between **3:50AM** and **3:55AM**

Set against the pre-engagement baseline of over four hours to a containment decision, this is the central proof that the engagement met its goal.

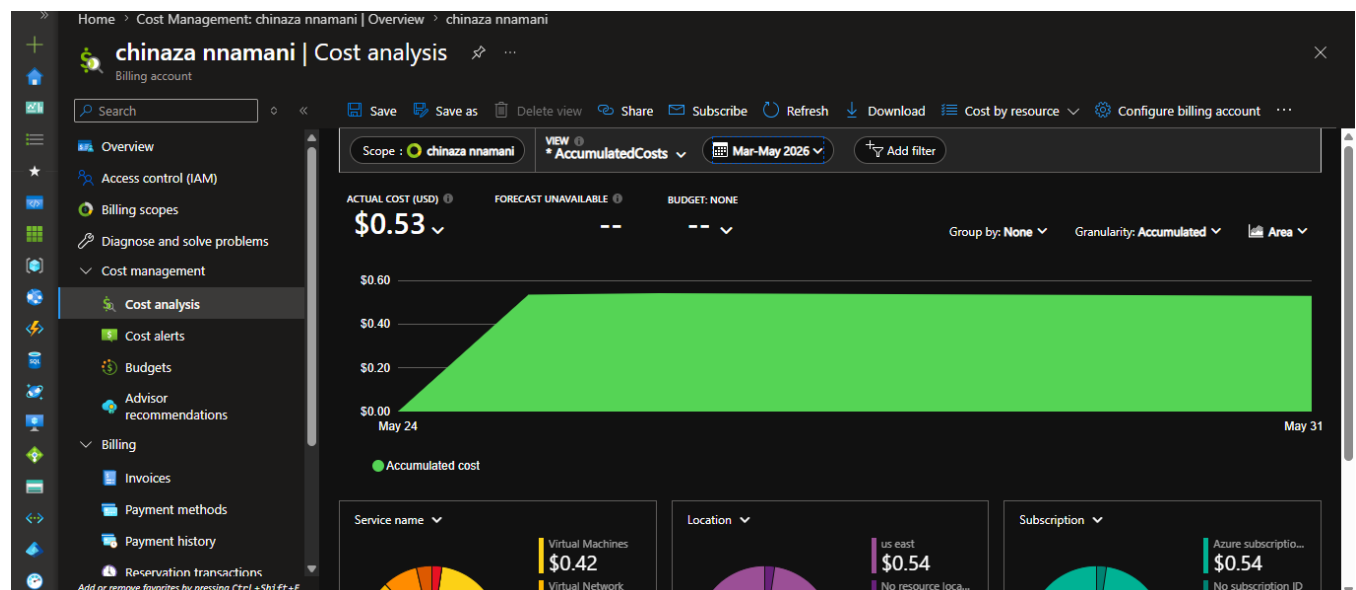
6. Results Against Success Metrics

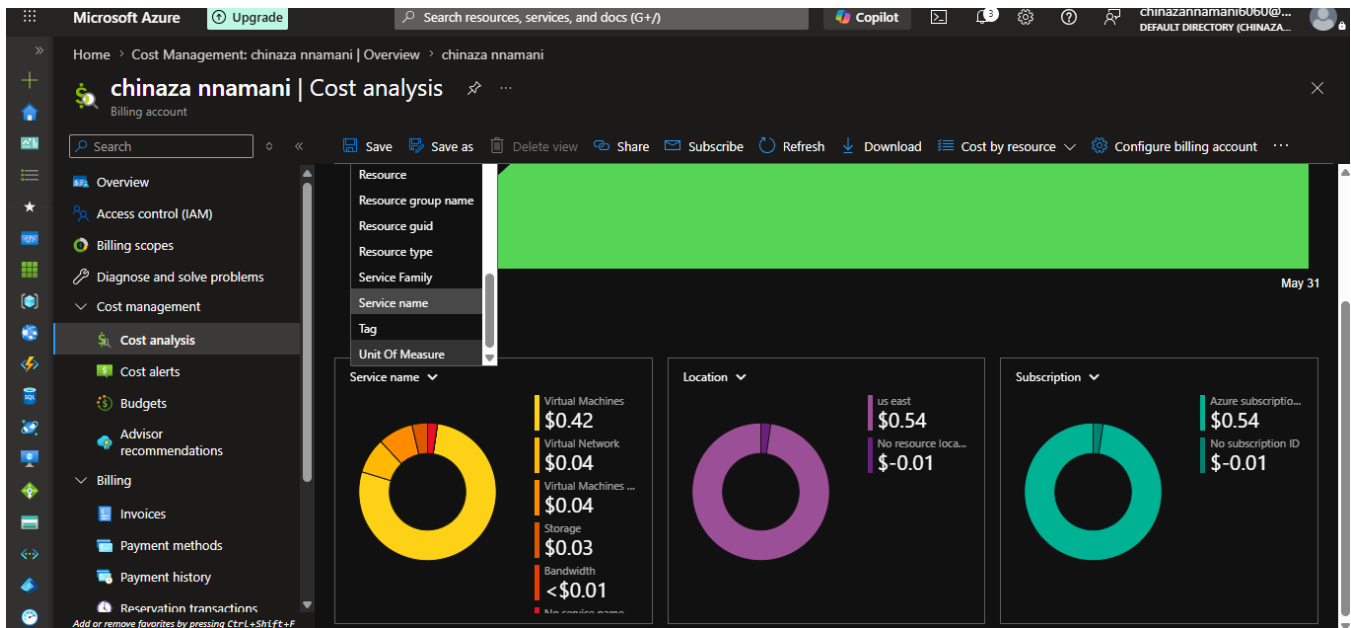
Success metric	Target	Outcome
Centralised visibility	100% of admin actions queryable within 15 minutes	Met
Analytics coverage	10+ rules enabled, all mapped to MITRE ATT&CK	Met
Custom detection	Privilege escalation rule fires on simulation	Met (after query fix)
Automated notification	Playbook fires within 2 minutes of incident	Met
Audit retention	Telemetry retained 90 days minimum	Met

7. Cost

Sentinel has a usage-based cost model with three components: data ingestion charged per gigabyte, data retention charged per gigabyte per month beyond the free period, and Logic Apps execution charged per action. Understanding this model is more useful than any single price, because prices change and usage does not.

For this proof of concept the running cost was close to zero, by design. Azure Activity logs ingest into a Log Analytics Workspace at no charge, the foundational Defender for Cloud tier is free, and the playbook ran only a handful of times, so Logic Apps consumption was negligible. The deliberate choice to defer the paid Defender plans is the main reason the proof of concept stayed near zero, and it is the main line item to plan for when moving to production.





Honesty note on figures

Exact per-gigabyte and per-action rates change over time and vary by region, so this report states the cost model rather than fixed numbers. Confirm current rates in the Azure Pricing Calculator before quoting a production budget.

8. Challenges and Lessons Learned

- **The friendly-name trap.** The biggest lesson was that the custom rule's text match on Owner and Contributor did not fire, because AzureActivity stores the role under the "Authorization" field.
The takeaway is permanent: always validate your detection against a real sample event before trusting it, and never assume a field contains the human-readable value you expect.
- **Verify ingestion first.** Running the take 10 queries before building detections saved time. A rule on an empty table fails silently, which is the worst kind of failure.
- **Test the whole chain, not the parts.** A rule that fires and a playbook that runs in isolation prove nothing about whether they work together. The single end-to-end test was worth more than testing each piece alone.
- **Name for clean-up.** Putting everything in one named resource group made the proof of concept easy to track and would make it easy to tear down. Small discipline, large payoff.
- **Navigating the Unified Microsoft Defender Portal.** Adapting to the newly unified Microsoft Defender portal interface required a steep learning curve. Locating where Microsoft Sentinel's incident management intersects with Defender's security alerts was initially confusing due to the vast layout of the defender portal.
- **Formulating the Optimal Logic App Playbook Workflow.** Finding the perfect workflow sequence for the automated response playbook presented significant trial and error.

9. Future Enhancements

The architecture is production-grade and extends by adding to it, not rebuilding it. In rough priority order:

1. Enable the paid Microsoft Defender for Cloud plans for deeper workload coverage.
2. Add a containment action to the playbook, for example automatically disabling a compromised account, behind an approval step.
3. Connect more data sources, starting with Microsoft Entra ID sign-in logs and Office 365.
4. Build an executive workbook so leadership can see posture at a glance.
5. Extend retention toward one hundred and eighty days for a stronger audit position.

10. Conclusion

This engagement changed CloudScale Logistics from a client that could not see across its own cloud to one that detects high-priority threats automatically and responds in minutes. The three-pillar approach, collect then identify then respond, gave the build a clear order and gives the client a clear path to grow. Most importantly for the client's immediate problem, the engagement produces the documented, demonstrable detection and response process that the ISO 27001 finding demanded.

For the analyst, the project is a compact tour of cloud security operations: workspace design, KQL detection engineering, MITRE mapping, and SOAR automation. The lesson that will outlast the project is the one the custom rule taught. Security work is not finished when something is built. It is finished when it has been tested against reality and written down so that someone else can trust it and rebuild it.

